

Conclusions i propostes de futur , Classificador de spam i phishing

1. Conclusions

Després de tot el procés de disseny, implementació i proves del classificador, estem bastant satisfets amb el que hem aconseguit. No ha estat un camí fàcil, però creiem que el resultat reflecteix bé les decisions tècniques que hem anat prenent al llarg del projecte i les raons que hi havia d'Aarrere de cadascuna.

Una de les primeres decisions que vam prendre i que ara veiem encertada va ser la d'utilitzar un ArbolBST per emmagatzemar el diccionari de paraules sospeschoses. La idea era clara: necessitàvem una estructura que ens permetés fer insercions i cerques de manera eficient, i el BST ens dona exactament això, amb un cost de $O(\log n)$ en el cas mitjà. Comparant-ho amb una llista lineal, que aniria a $O(n)$, la millora és evident, sobretot quan el volum de paraules creix. A mEés, vam decidir integrar el recompte de freqüència directament al node, cosa que ens va evitar haver de fer recorreguts addicionals per l'arbre cada vegada que volíem actualitzar una comptabilitat. En resum, una decisió senzilla però que ens va estalviar bastant de maldecap.

```
def _insertar(self, nodo, palabra, peso):
    # caso base: posición vacía, creamos el nodo aquí
    if nodo is None:
        return NodoBST(palabra, peso)
    if palabra < nodo.palabra:
        nodo.izquierdo = self._insertar(nodo.izquierdo, palabra, peso)
    elif palabra > nodo.palabra:
        nodo.derecho = self._insertar(nodo.derecho, palabra, peso)
    else:
        # ya estaba en el árbol, solo sumamos una aparición más
        nodo.frecuencia += 1
    return nodo

def buscar(self, palabra):
    return self._buscar(self.raiz, palabra.lower())

def _buscar(self, nodo, palabra):
    if nodo is None or nodo.palabra == palabra:
        return nodo
    if palabra < nodo.palabra:
        return self._buscar(nodo.izquierdo, palabra)
    return self._buscar(nodo.derecho, palabra)

def calcular_riesgo_total(self):
    return self._calcular_riesgo(self.raiz)

def _calcular_riesgo(self, nodo):
    if nodo is None:
        return 0
    izq = self._calcular_riesgo(nodo.izquierdo)
    actual = nodo.peso * nodo.frecuencia
    der = self._calcular_riesgo(nodo.derecho)
    return izq + actual + der
```

Una altra cosa de la qual estem orgullosos és la dtecció de typosquatting mitjançant la

distància de Levenshtein. Quan vam veure per primera vegada exemples com 'santanderr.com' o 'g00gle.com', vam tenir clar que una comparació exacta de cadenes no era suficient. Qualsevol sistema que no capturi aquestes imitacions visuals és un sistema que falla en un dels vectors d'atac més comuns del phishing actual. Vam estar una bona estona calibrant el llindar i finalment vam optar per ≤ 2 , que equilibra prou bé la sensibilitat amb el nombre de falsos positius. Si el posàvem massa alt, el filtre es tornava paranoic i marcava coses que no calia; si el posàvem massa baix, deixava passar coses que sí que eren perilloses.

```
3     return dp[m][n]
4
5     # comprueba si un dominio es una imitación falsa de uno legítimo
6     def es_typosquatting(dominio):
7         # si la distancia es <= 2 significa que solo hay 1 o 2 caracteres de diferencia, sospechoso
8         dominio = dominio.lower().strip()
9         for legitimo in DOMINIOS_LEGITIMOS:
10             if dominio == legitimo:
11                 return False # es el mismo no pasa nada
12             if distancia levenshtein(dominio, legitimo) <= 2:
13                 return True
14         return False
15
16
17     def extraer_dominios(texto):
18         # busca dominios en URLs y también en direcciones de email
19         patron = r'https?:\/\/([a-zA-Z0-9.\-]+)'
20         dominios = re.findall(patron, texto)
21         patron_email = r'@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}'
22         dominios += re.findall(patron_email, texto)
23         return dominios
24
```

Quant a l'arquitectura general, vam prendre la decisió de dissenyar el sistema al voltant d'una classe base FiltroCorreo que actua com a interfície comuna per a tots els filtres. Això ens va permetre afegir nous filtres al sistema sense haver de tocar AnalizadorCorreo, seguint el principi obert/tancat (OCP). Quan ho vam implementar, ens vam adonar que era una d'aquelles decisions que semblen una mica excessives al principi "per a un projecte d'uni, tot això?" però que al final donen molt de joc i fan el codi molt més net i mantenible. Si algun dia afegim un filtre per a HTML o un per a adjunts, senzillament creem una nova subclasse i ja està, sense trencar res del que ja funciona.

El sistema de puntuació ponderat també va ser una part en la qual vam invertir temps. Vam veure de seguida que no tenia cap sentit tractar igual la paraula 'cvv' que la paraula 'gratis'. La primera apareix gairebé exclusivament en correus fraudulents que intenten robar dades bancàries; la sgona és sospeschosa però pot aparèixer en un newsletter legítim d'una botiga. Assignar pesos diferenciats 65 punts per a 'cvv', 25 per a 'gratis', per exemple fa que la classificació final sigui molt més matisada i útil. El fet de tenir tres nivells de classificació SEGURO, SOSPECHOSO i PELIGROSO en lloc d'un simple binari sí/no ens va semblar una gran millora, perquè dóna flexibilitat a l'hora d'actuar: un correu SOSPECHOSO potser el poses en una carpeta de revisió, mentre que un PELIGROSO el

bloqueges directament.

Finalment, la funció `escanear_carpeta()` amb recorregut recursiu va ser una de les parts més pràctiques del projecte. Els clients de correu reals no guarden tots els missatges en una única carpeta plana; tenen estructures niuades amb subcarpetes de Rebuts, Arxiu, per anys, per remittents... La recursivitat ens permet tractar-ho de manera transparent i sense afegir complexitat extra al codi principal.

```
def escanear_carpeta(carpeta, analizador):
    # Aquí recorre una carpeta y todas sus subcarpetas buscando archivos .eml
    resultados = []

    try:
        entradas = os.listdir(carpeta)
    except Exception as e:
        print(f"No se puede leer la carpeta {carpeta}: {e}")
        return resultados

    for entrada in entradas:
        ruta_completa = os.path.join(carpeta, entrada)

        if os.path.isdir(ruta_completa):
            # es una subcarpeta -> llamada recursiva
            resultados += escanear_carpeta(ruta_completa, analizador)

        elif os.path.isfile(ruta_completa) and entrada.lower().endswith(".eml"):
            # es un correo .eml -> analizarlo
            resultado = analizador.analizar_eml(ruta_completa)
            resultados.append(resultado)

    return resultados
```

2. Limitacions actuals

Ser honestos sobre les limitacions del que hem fet ens sembla igual d'important que explicar el que funciona. I n'hi ha alguna que tenim

La més evident a nivell estructural és que el BST que hem implementat no és balancejat. Això significa que, si les paraules s'insereixen en ordre alfabètic cosa que passa si, per exemple, les carreguem d'un fitxer ordenat, l'arbre degenera en una llista encadenada i tornem a $O(n)$. Mentre el sistema funcioni amb diccionaris petits i en un entorn controlat, no es nota, però en un escenari real seria un problema seriós de rendiment.

Una altra limitació que ens fa una mica de mandra reconèixer és que tot el diccionari de paraules spam i tots els dominis legítims estan hardcodejats directament al codi. Durant el desenvolupament és còmode, però en un sistema real seria inacceptable: qualsevol actualització requeriria modificar el codi font, recompilar i redespregar. Els atacants canvien les seves tàctiques constantment; un diccionari estàtic és un sistema que

envelleix molt ràpid.

I per últim, els falsos positius. Paraules com 'transferencia' o 'contraseña' poden aparèixer perfectament en un correu legítim del teu banc o del teu employer. Hem intentat calibrar els pesos per minimitzar-ho, però és un problema inherent a qualsevol sistema basat en paraules clau sense context semàntic.

3. Propostes de futur

Si seguíssim treballant en aquest projecte, hi ha un conjunt clar de millores que implementaríem.

La primera i més urgent seria substituir el BST actual per un arbre AVL o un Red-Black Tree. Tots dos garanteixen un balanceig automàtic que manté la profunditat de l'arbre a $O(\log n)$ fins i tot en el pitjor cas d'ordre d'inserció. No és un canvi d'implementar, però l'impacte en el rendiment quan el sistema escala és enorme. Un Red-Black Tree, en concret, és el que utilitzen internament moltes implementacions de maps i diccionaris en llenguatges com Java o C++, precisament per aquesta garantia de rendiment.

La segona millora seria afegir nous filtres per cobrir vectors d'atac moderns. Un FiltroPorHTML que parsegi el contingut HTML del correu amb BeautifulSoup i extregui text, links i atributs sospitosos. Un FiltroPorAdjunts que comprovi les extensions dels fitxers adjunts i llenci un avís quan detecti .exe, .js, .vbs, .bat o fitxers comprimits que continguin executables.

I per tancar, dues millores més pràctiques serien una interfície gràfica senzilla pot ser Tkinter o una aplicació web lleugera per visualitzar els resultats d'una manera accessible per a usuaris no tècnics, i una suite de tests automàtics amb pytest que cobreixi cada filtre individualment i el sistema complet amb correus .eml reals etiquetats. Ara mateix el testing és manual, i això és un risc: qualsevol refactorització pot introduir errors que no detectem fins que ho provem a mà.

4. Valoració final

Mirant enrere, creiem que el projecte ha assolit l'objectiu principal que ens vam marcar des del principi: construir un sistema capaç de classificar correus .eml detectant spam i phishing de manera eficient, combinant cerca en arbre binari amb similitud de cadenes per capturar imitacions de domini. No és perfecte i hem sigut bastant explícits sobre on falla, però és un sistema funcional, ben estructurat i, sobretot, extensible.

L'arquitectura orientada a objectes amb herència i polimorfisme ha demostrat ser una bona elecció. El fet que puguem afegir nous filtres sense tocar el nucli del sistema és

exactament el tipus de disseny que s'ensenya a teoria i que sovint costa veure aplicat en un projecte real. En aquest cas, nosaltres mateixos hem pogut comprovar com facilita el desenvolupament incremental.

Les principals àrees de millora que identifiquem el balanç de l'arbre, la cobertura de vectors d'atac moderns com HTML i adjunts, i la incorporació d'aprenentatge automàtic no són crítiques per a un prototip com el que hem fet, però sí que serien essencials si el sistema hagués d'operar en un entorn real. Amb les propostes de futur que hem wdescrit, el camí cap a un sistema de producció robust està bastant clar.

En definitiva, estem contents amb el que hem entregat, hem après molt i si haguéssim de tornar a fer-ho, probablement prendríem les mateixes decisions estructurals però amb un ull posat des del principi en el tema del balanç de l'arbre i l'externalització del diccionari.